Reviewer Pack — Free Entropy Lower Bound on Disjointness Communication Compl…

Entry 9020cef6c62f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 18:58:09 UTC

Free Entropy Lower Bound on Disjointness Communication Complexity

Entry ID: `9020cef6c62f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 18:58:09 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For any n ≥ 1, the free entropy ϕ(M_n) of the DISJOINTNESS matrix M_n ∈ {0,1}^{n×n} satisfies ϕ(M_n) ≥ Ω(log n). This bound is tight up to constant factors for random instances.

Rationale (proposer's reasoning):

> Free entropy captures the 'non-commutative complexity' of random variables. By analyzing the DISJOINTNESS matrix's entries as non-commutative random variables, we may uncover structural constraints that mirror communication complexity lower bounds. The logarithmic scaling aligns with known Ω(n) lower bounds via Forster's theorem.

Taxonomy category: `COMMUNICATION_COMPLEXITY` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `18469b2cedbb104e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate entries below the pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def determinant(A):
    n = len(A)
    det = 1
    for i in range(n):
        det *= A[i][i]
    return det

def free_entropy(M, n):
    def log_det_tM(t):
        I_plus_tM = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                I_plus_tM[i][j] = M[i][j] * t + (i == j)
        return math.log(determinant(gaussian_elimination(I_plus_tM)))

    integral = 0
    dt = 0.01
    for t in [dt*i for i in range(1, int(1/dt))]:
        integral += log_det_tM(t) * dt / t
```

```
    return -integral

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    M_n = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    phi_M_n = free_entropy(M_n, n)
    conjecture_holds = phi_M_n >= math.log(n) * 0.9
    counterexample = "" if conjecture_holds else "free entropy < log(n)"

    return {
        "metric_name": "Free Entropy",
        "metric_value": phi_M_n,
        "instances_tested": n*n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}**}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3542cd42.py", line 79, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3542cd42.py", line 61, in run_trial
    phi_M_n = free_entropy(M_n, n)
              ^^^^^^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3542cd42.py", line 53, in free_entropy
    integral += log_det_tM(t) * dt / t
                ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3542cd42.py", line 48, in log_det_tM
    return math.log(determinant(gaussian_elimination(I_plus_tM)))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with math domain error, preventing result verification | next: Debug numerical stability in free_entropy computation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	68547
2	propose	ollama_remote	qwen3:8b	0	0	112334
3	propose	ollama_remote	qwen3:8b	0	0	110367
4	preregistration	ollama_remote	qwen3:8b	0	0	24189
5	novelty	ollama_remote	qwen3:8b	0	0	20687
6	novelty	ollama_remote	qwen3:8b	0	0	19374
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13822
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8885
9	judge	ollama_remote	qwen3:8b	0	0	16213

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 394419 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9020cef6c62f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9020cef6c62f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9020cef6c62f.tar.gz` (if generated)